Project Report

Object Detection and Tracking Implementation

This document outlines the methodology, challenges, and final approach for implementing a multi-object tracking system for sports analytics, detailing the evolution from standard libraries to a custom-built solution.

1. Initial Goal & Approach

The primary objective was to build a robust tracking system capable of identifying and following multiple objects (players, referees, ball) in a video with minimal identity (ID) swapping. The project began by leveraging standard, widely-used components before progressing to more specialized techniques.

2. Techniques Tried and Their Outcomes

Our development process was iterative, moving from simple configurations to more complex algorithms as we worked to solve the core challenges of object tracking in a dynamic sports environment.

Stage 1: Standard YOLOv8 Trackers (ByteTrack & BoT-SORT)

- **Technique:** Our first step was to use the official, built-in trackers provided with YOLO, such as bytetrack.yaml and botsort.yaml. These trackers are fast and effective for many general-purpose scenarios. We attempted to optimize their performance by tuning key hyperparameters:
  - track_buffer: Increased the number of frames an object ID is kept alive after being lost, to handle short-term occlusions.
  - track_high_thresh / track_low_thresh: Adjusted the confidence thresholds for initiating and terminating tracks.
  - frame_rate: Ensured the tracker's internal timing matched the video's frame rate.
- **Outcome: Insufficient.** While these trackers performed reasonably well, they struggled in complex scenes with frequent occlusions and fast-moving, similar-looking players. This resulted in a high number of ID swaps, which was unacceptable for reliable player analysis. This led us to explore more advanced, appearance-based trackers.

Stage 2: Advanced Library-Based Trackers (DeepSORT & StrongSORT)

- **Technique:** To improve identity preservation, we moved to algorithms that use appearance features. Our goal was to integrate a powerful Re-ID model (OSNet) using the deep-sort-realtime Python library and the related StrongSORT algorithm. We tried several integration methods:
    1. **Direct Model Passing:** Passing a manually loaded OSNet model to the tracker.
    2. **Custom Wrapper Class:** Creating a dedicated class to handle inference and transformations.
- **Built-in torchreid Loader:** Using the library's intended method for loading torchreid-compatible models.
- **Outcome: Failure.** All attempts in this stage failed due to a critical **library version incompatibility**. The installed version of deep-sort-realtime was significantly outdated, and its API did not support any of the modern methods for integrating a custom Re-ID model. This led to a series of TypeError and Exception messages, making it impossible to proceed with these libraries without a forced environment update.

3. Challenges Encountered

- **ID Swapping:** The initial standard trackers (ByteTrack, BoT-SORT) were not robust enough to handle the specific challenges of tracking athletes, leading to frequent identity errors.
- **Library Incompatibility:** The primary blocker for using more advanced off-the-shelf solutions like DeepSORT was an outdated and inflexible library version, which prevented the integration of our superior Re-ID model.

4. The Final, Successful Approach: A Custom RAFT-Enhanced Tracker

Given the limitations of existing libraries, we pivoted to building our own tracker from the ground up, giving us full control over the logic and allowing us to combine multiple state-of-the-art techniques.

- **Methodology:** The final code implements a sophisticated tracker that fuses three powerful concepts:
    1. **Detection (YOLOv11):** Continues to serve as the high-quality source for object locations in each frame.

2. **Motion Prediction (Kalman Filter + RAFT Optical Flow):** A **Kalman Filter** predicts an object's future position based on its past motion. This is significantly enhanced by **RAFT Optical Flow**, which analyzes pixel-level motion between frames to provide a highly accurate, short-term motion vector, making the tracker more resilient to abrupt movements.
3. **Appearance Matching (ResNet18 Re-ID):** For handling longer occlusions or re-acquiring lost tracks, we use a **ResNet18** model to extract an "appearance embedding" from each object. By comparing the **cosine distance** of these embeddings, the tracker can correctly re-identify a player even after they have been hidden for several frames.

- **Outcome: Significant Improvement.** This custom approach proved to be the most effective strategy. While no tracker is perfect in highly complex scenarios, this solution has **resolved an estimated 75%+ of the ID swapping issues** encountered with the initial trackers. It successfully circumvents the library dependency problems and results in a far more stable and reliable tracking system tailored specifically to our needs.